

3. Prepare Test Plan

3.1 Introduction

A Test Plan is a formal document that defines the testing strategy, objectives, scope, resources, schedule, risks, deliverables, and overall approach for testing a software application. It serves as a roadmap for the testing team and ensures that all testing activities are planned, organized, and executed systematically.

For this project, a Test Plan has been prepared for the **Parabank Banking Application** to validate the application's functionality through manual and automated testing. The project also includes CI/CD pipeline implementation using GitHub Actions to support continuous testing and rapid feedback.

The Test Plan helps ensure complete test coverage of business requirements and provides a structured approach to quality assurance activities.

3.2 Project Information

Item	Details
Project Name	Parabank Automation Framework
Application Name	Parabank Banking Application
Application Type	Web-Based Banking Application
Testing Type	Functional & Automation Testing
Automation Tool	Selenium WebDriver
Programming Language	Java
Framework	TestNG
Build Tool	Maven
Version Control	Git
CI/CD Tool	GitHub Actions
Test Management	Excel-based Documentation

3.3 Test Plan Objectives

The primary objectives of this testing activity are:

Functional Validation

- Verify that all application modules function according to business requirements.
- Ensure banking operations are executed correctly.
- Validate data integrity across modules.

Automation Validation

- Develop reusable automation scripts.
- Validate automation framework stability.
- Ensure data-driven execution works correctly.

CI/CD Validation

- Verify automated execution through GitHub Actions.
- Ensure tests execute successfully after every code commit.
- Validate automated report generation.

Quality Assurance Objectives

- Identify defects early.
 - Reduce regression risks.
 - Improve software reliability.
 - Increase test coverage.
-

3.4 Testing Scope

In Scope

The following modules are included in testing:

User Registration

- New user registration
- Field validations
- Duplicate username validation
- Registration confirmation

Login

- Valid login
- Invalid login
- Blank field validation
- Error message validation

Account Overview

- Account visibility
- Balance validation
- Account details verification

Open New Account

- Savings account creation
- Checking account creation
- Account visibility validation

Transfer Funds

- Fund transfer functionality
- Balance validation
- Transfer confirmation

Bill Payment

- Bill payment process
- Payee information validation
- Payment confirmation

Find Transactions

- Search by Transaction ID
- Search by Date
- Search by Date Range
- Search by Amount

Request Loan

- Loan request submission
- Loan approval validation
- Loan rejection validation

Update Contact Information

- Address update
- Phone update
- ZIP code update
- Data persistence validation

Out of Scope

The following areas are not included:

- Mobile Application Testing
- API Testing
- Database Testing
- Performance Testing
- Security Testing

- Load Testing
 - Stress Testing
 - Accessibility Testing
-

3.5 Test Strategy

The testing strategy defines how testing activities will be performed.

Manual Testing

The following activities will be completed manually:

- Requirement Analysis
 - Test Scenario Creation
 - Test Case Design
 - Defect Reporting
 - Requirement Traceability Matrix Preparation
-

Automation Testing

The following activities will be automated:

- Login Testing
- Account Overview Testing
- Fund Transfer Testing
- Bill Payment Testing

Automation will be implemented using:

- Selenium WebDriver
 - Java
 - TestNG
 - Maven
 - Page Object Model
-

Data-Driven Testing

Test data will be maintained in Excel files.

Examples:

- LoginData.xlsx
- TransferFundsData.xlsx
- BillPayData.xlsx

Benefits:

- Easy maintenance
 - Better test coverage
 - Reduced code duplication
-

3.6 Test Approach

The following testing approach will be followed:

Requirement Analysis

Review business requirements and identify testable features.

Test Planning

Define testing objectives, scope, resources, and schedule.

Test Design

Create:

- Test Scenarios
- Test Cases
- RTM

Test Automation Development

Develop automation scripts using Page Object Model.

Test Execution

Execute:

- Manual Test Cases
- Automated Test Cases

Defect Management

Log, track, and retest identified defects.

Test Closure

Prepare final testing reports and closure documents.

3.7 Test Environment

Hardware Requirements

Component	Specification
Processor	Intel i5 or Higher
RAM	8 GB Minimum
Storage	20 GB Free Space

Software Requirements

Software	Version
Windows OS	Windows 10/11
Java	JDK 21
Eclipse IDE	Latest
Maven	Latest
Git	Latest
Google Chrome	Latest
ChromeDriver	Compatible Version

3.8 Automation Framework Architecture

The framework follows Page Object Model architecture.

Framework Components

Base Layer

Contains:

- BaseTest.java
- DriverFactory.java

Responsibilities:

- Browser initialization
- Driver management

- Test setup and teardown
-

Utility Layer

Contains:

- ConfigReader.java
- ExcelReader.java
- ScreenshotUtil.java

Responsibilities:

- Configuration management
 - Excel data handling
 - Screenshot capture
-

Page Layer

Contains:

- LoginPage.java
- HomePage.java
- AccountPage.java
- TransferPage.java
- BillPage.java

Responsibilities:

- Web element identification
 - Page actions
-

Test Layer

Contains:

- LoginTest.java
- AccountTest.java
- TransferFundsTest.java
- BillPayTest.java

Responsibilities:

- Test execution

- Assertions
 - Validation
-

3.9 Test Deliverables

The following deliverables will be produced during the project.

Documentation Deliverables

- Project Introduction
 - Requirement Analysis
 - Test Plan
 - Test Scenarios
 - Test Cases
 - RTM
 - Environment Setup Document
-

Automation Deliverables

- Automation Framework
 - Selenium Scripts
 - Test Data Files
 - GitHub Repository
-

Execution Deliverables

- Execution Report
 - Defect Report
 - Screenshots
 - TestNG Reports
 - Extent Reports
-

CI/CD Deliverables

- GitHub Actions Workflow
 - Pipeline Documentation
 - Pipeline Execution Reports
-

3.10 Roles and Responsibilities

Role	Responsibility
QA Engineer	Requirement Analysis
QA Engineer	Test Scenario Creation
QA Engineer	Test Case Design
QA Engineer	Automation Script Development
QA Engineer	Defect Reporting
QA Engineer	CI/CD Integration
QA Engineer	Report Preparation

3.11 Entry Criteria

Testing will begin only when:

- Requirements are available.
 - Application is accessible.
 - Test environment is ready.
 - Test data is prepared.
 - Automation framework setup is completed.
-

3.12 Exit Criteria

Testing will be considered complete when:

- All planned test cases are executed.
 - Critical defects are resolved.
 - Test reports are generated.
 - Automation execution is successful.
 - CI/CD pipeline execution is successful.
 - Test closure report is prepared.
-

3.13 Defect Management Process

The following process will be followed:

1. Defect Identification
2. Defect Logging
3. Defect Assignment

4. Defect Fixing
5. Retesting
6. Defect Closure

Defect severity levels:

Severity	Description
Critical	Application crash or major failure
High	Major functionality impacted
Medium	Partial functionality impacted
Low	Minor UI or cosmetic issue

3.14 Risk Analysis

Risk ID	Risk Description	Impact
R001	Application downtime	High
R002	Browser compatibility issues	Medium
R003	Test data inconsistency	Medium
R004	UI changes	High
R005	Network failures	Medium
R006	GitHub Actions workflow failure	High

3.15 Risk Mitigation Plan

Risk	Mitigation
Application Downtime	Schedule testing during stable periods
UI Changes	Use maintainable locators
Test Data Issues	Maintain dedicated test data
CI/CD Failures	Validate workflow before deployment
Browser Issues	Regular compatibility testing

3.16 Test Execution Schedule

Activity	Estimated Duration
Requirement Analysis	1 Day
Test Planning	1 Day
Test Scenario Creation	2 Days
Test Case Design	3 Days

Activity	Estimated Duration
Automation Development	5 Days
CI/CD Setup	2 Days
Test Execution	2 Days
Defect Reporting	1 Day
Test Closure	1 Day

3.17 Success Criteria

The project will be considered successful if:

- All planned modules are tested.
 - Automation scripts execute successfully.
 - CI/CD pipeline runs without failures.
 - Test reports are generated automatically.
 - Defects are properly documented.
 - Project deliverables are completed.
-

3.18 Conclusion

This Test Plan provides a structured roadmap for testing the Parabank Banking Application through both manual and automated testing approaches. The plan defines the testing scope, objectives, resources, deliverables, risks, and execution strategy required to ensure software quality. By integrating Selenium automation with GitHub Actions CI/CD, the project enables continuous testing, faster feedback, improved reliability, and enhanced software delivery processes. This document serves as the foundation for all subsequent testing activities and ensures a systematic approach to achieving the project's quality objectives.